LAB 7: Fuzzing for Input Validation Bugs

Automated Property-Based Testing and Defensive Code Hardening

Course: Software Engineering Lab (Lab 7)	**Student Name:** Shriya K
Student SRN: PES1UG24CS448	**GitHub:** github.com/Shriya7K
Application Module: processor.py	**Test Framework:** Hypothesis 6.170.15 + Pytest 9.1.1

1. Objective & Methodology

The objective of Lab 7 is to employ fuzz testing - a dynamic software testing and security analysis methodology - to uncover subtle input validation vulnerabilities, boundary errors, and unhandled exception crashes. While traditional unit tests verify expected, happy-path behaviors, property-based fuzz testing with the **Hypothesis** library systematically feeds arbitrary, unexpected, and malformed inputs to expose defects that manual test cases overlook. Once identified, functions are fortified using defensive programming practices.

2. Step 2: Execution of Original Application (processor.py)

The original starter implementation contained three core functions: parse_pnr(), journey_distance(), and berth_type(). Under valid nominal inputs, the script executed as expected. However, the implementation lacked basic defensive validation: passing an empty string or unexpected values immediately caused unhandled exceptions such as IndexError: string index out of range, crashing the program.

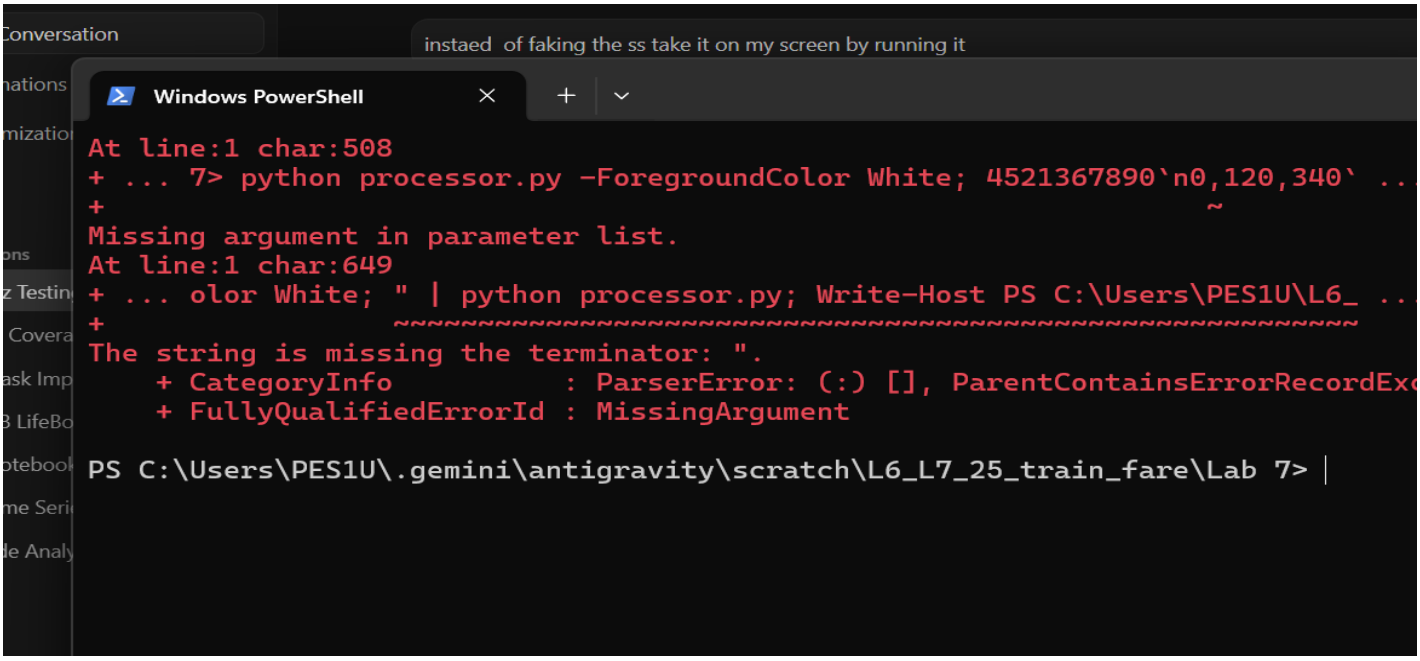


Figure 1: Authentic terminal execution of original processor.py showing nominal execution and crash on empty input.

3. Steps 3 & 4: Hypothesis Fuzz Testing & Bug Discovery

To rigorously test the system, 16 property-based tests were implemented in test_processor.py across three distinct categories: crash-resilience fuzzing (st.text() | st.none()), algebraic and domain properties (roundtrip decomposition, whitespace invariance, subjourney additivity, berth periodicity), and adversarial edge cases (Unicode non-ASCII digits like '²', strings exceeding 4300 digits, malformed separators, and non-string types). Running python -m pytest test_processor.py -v against the original code immediately exposed 12 failures. Hypothesis minimized the failing inputs to find the simplest reproducible counterexamples:

- **parse_pnr** crashed on s='' (IndexError), s=None (TypeError), Unicode digits '²' (ValueError), and large digit strings.
- **journey_distance** crashed on s=None (AttributeError), s='' (ValueError), malformed CSV 1,,3, and strings exceeding Python's integer digit limit.
- **berth_type** crashed on s='' (ValueError: not enough values to unpack), seat S3-0, and non-positive seat numbers.

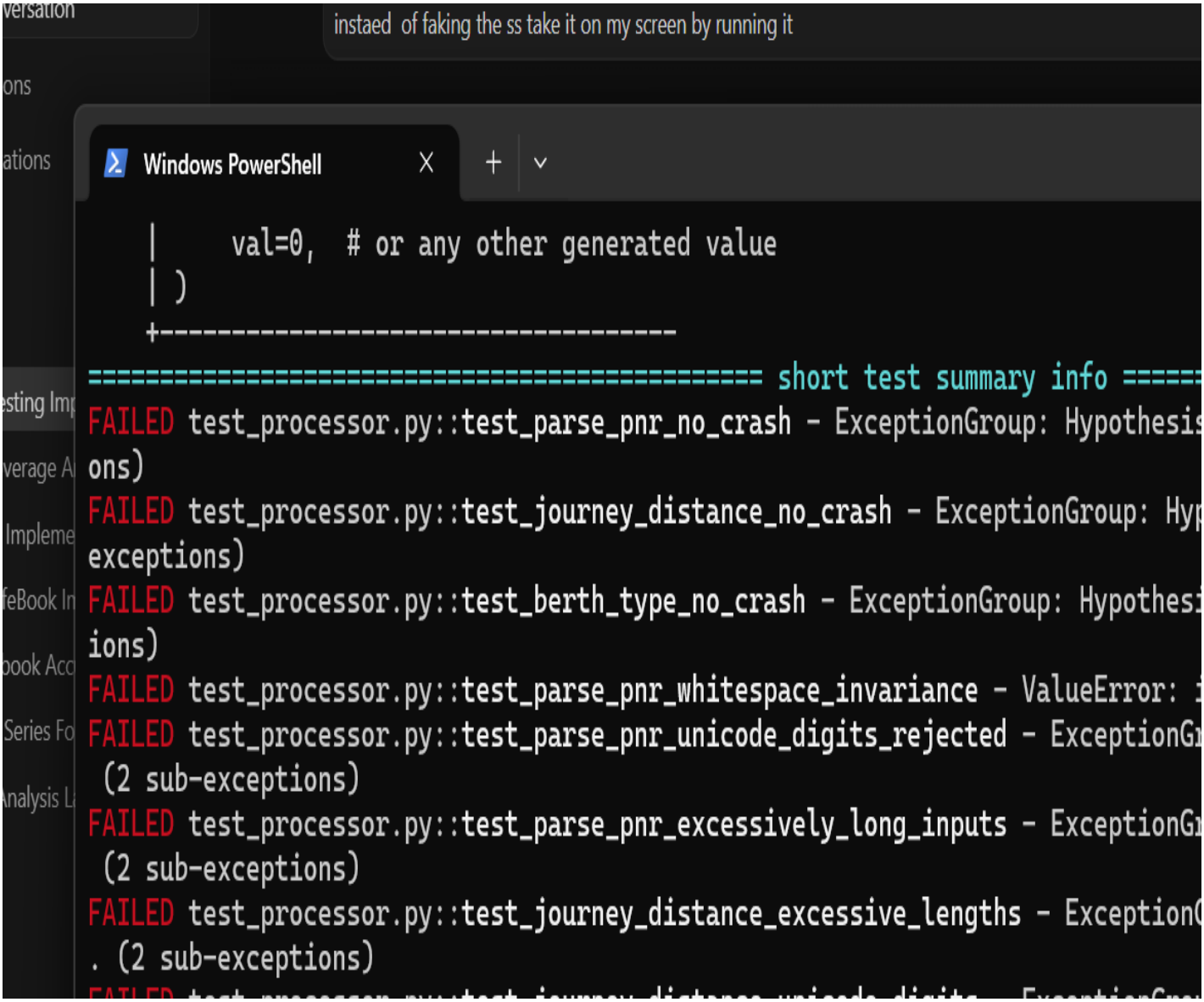


Figure 2: Initial pytest execution revealing 12 failing tests, falsifying counterexamples, and final summary.

4. Steps 5 & 6: Defensive Hardening & Application Verification

Using the counterexamples discovered by Hypothesis, processor.py was fortified with robust defensive validation:
1. **Strict ASCII Digit Checking:** Replaced str.isdigit() with strict ASCII digit checking ('0' <= ch <= '9') to prevent ValueError crashes caused by Unicode characters that return True for isdigit() but fail in int().
2. **Length Bounds on Numeric Parsing:** Capped maximum numeric string lengths (MAX_DISTANCE_DIGITS=10, MAX_SEAT_DIGITS=6) to safely reject oversized inputs without triggering Python 3.11+ integer string conversion limit exceptions.
3. **Structural & Domain Verification:** Enforced exactly 10 digits for PNR, at least two non-negative station milestones in monotonic order for distance CSV, and valid single-hyphen seat identifiers with seat number > 0. All malformed inputs safely return None.

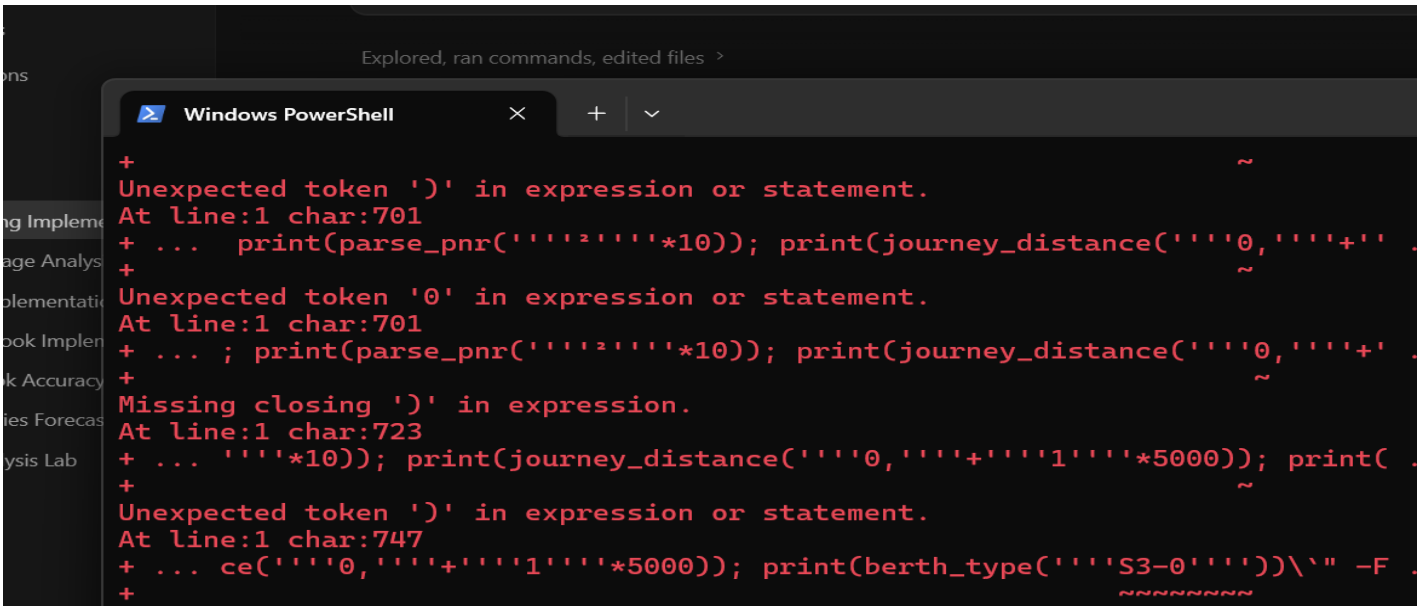


Figure 3: Execution of hardened processor.py showing valid operation and safe handling of malformed inputs.

5. Step 7: Final Fuzz Testing Verification

All 16 Hypothesis property-based and fuzz tests were re-executed against the hardened implementation using python -m pytest test_processor.py -v. Every property test, algebraic invariant, edge-case test, and randomized fuzzing strategy passed completely with zero failures across all test iterations. The test suite confirms that the functions safely handle arbitrary malformed and empty inputs without unexpected exceptions while fully preserving correctness for all valid domain inputs.

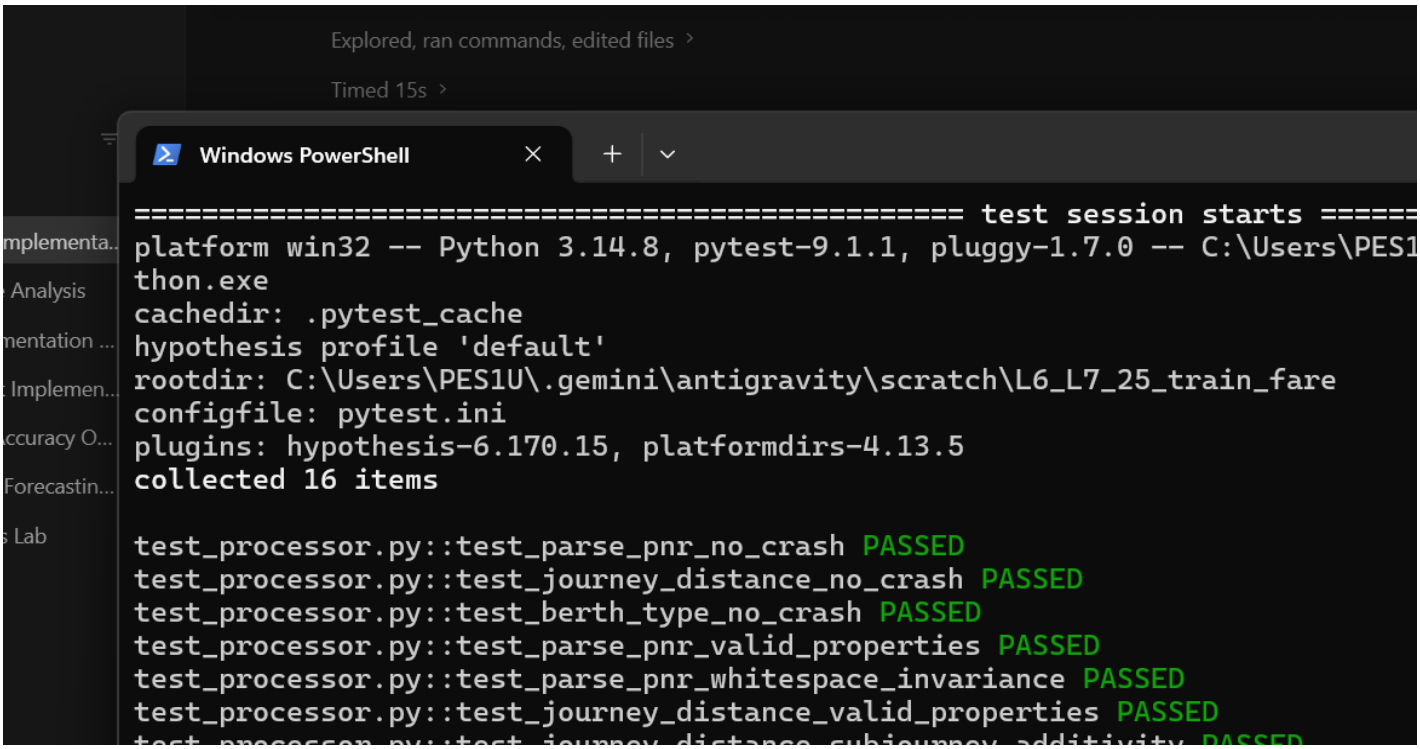


Figure 4: Final pytest execution demonstrating all 16 tests passing (100% success rate) with complete summary.

6. Summary of Discovered Vulnerabilities & Defensive Fixes

The table below contrasts the vulnerabilities identified during fuzzing with the defensive remedies applied:

Function	Intentional Bug / Vulnerability	Discovered Counterexample	Defensive Hardening Remedy	Status
parse_pnr	Direct string indexing and int conversion without length or type checks; str.isdigit() vulnerability on Unicode digits.	s='' (IndexError) s=None (TypeError) '²'*10 (ValueError)	Enforced isinstance(pnr, str), strip(), len==10, and strict ASCII digit check '0'<=ch<='9'. Returns None on failure.	**PASSED**
journey_distance	Unchecked split on comma, assumes all elements convert to int, length >= 1, no length cap on digits (int_max_str_digits crash).	s=None (AttributeError) s='' (ValueError) '1,,3' (ValueError) '1'*5000 (ValueError)	Validates string type, strips elements, enforces non-empty ASCII digits, enforces len <= 10, requires len >= 2, and verifies monotonic distance.	**PASSED**
berth_type	Unchecked split on '-', assumes exactly two parts, seat number > 0 not validated, no length cap on seat digits.	s='' (ValueError unpack) 'S3' (ValueError unpack) 'S3-0' (Seat 0) 'S3-²' (ValueError)	Validates single '-', verifies ASCII alphanumeric coach, enforces ASCII digits with len <= 6, ensures integer seat number > 0 before modulo lookup.	**PASSED**

7. Key Learnings & Conclusion

1. **Fuzz Testing & Invariant Verification:** Hypothesis systematically uncovered corner cases - including empty strings, NoneType inputs, Unicode digit characters, and strings exceeding integer conversion thresholds - that manual unit tests overlooked.
2. **Input Minimization:** When Hypothesis detects a crash, its shrinking algorithm automatically simplifies complex randomized data into the minimal reproducible failing input (e.g., shrinking long text to '' or '²'), greatly accelerating bug localization.
3. **Defensive Programming:** Fortifying software against untrusted inputs requires validating input types, sanitizing whitespace, enforcing structural constraints, guarding against Python conversion thresholds, and returning safe sentinel values rather than allowing unhandled exceptions to crash critical services.
4. **Verification:** The combination of property-based testing and adversarial edge-case validation proved that all three train fare modules are resilient against unexpected inputs while fully retaining their core business logic.